

Try Mermaid Advanced Editor — OSS users get 10% off with code JS26

Get started

Flowcharts - Basic Syntax

Flowcharts are composed of **nodes** (geometric shapes) and **edges** (arrows or lines). The Mermaid code defines how nodes and edges are made and accommodates different arrow types, multi-directional arrows, and any linking to and from subgraphs.

WARNING

If you are using the word "end" in a Flowchart node, capitalize the entire word or any of the letters (e.g., "End" or "END"), or apply this [workaround](#). Typing "end" in all lowercase letters will break the Flowchart.

WARNING

If you are using the letter "o" or "x" as the first letter in a connecting Flowchart node, add a space before the letter or capitalize the letter (e.g., "dev--- ops", "dev---Ops").

Typing "A---oB" will create a [circle edge](#).

Typing "A---xB" will create a [cross edge](#).

A node (default)

Code:

```
----
title: Node
----
flowchart LR
    id
```

mermaid

⌘ + Enter | [Run ▶](#)

INFO

The id is what is displayed in the box.

TIP

Instead of `flowchart` one can also use `graph` .

A node with text

It is also possible to set text in the box that differs from the id. If this is done several times, it is the last text found for the node that will be used. Also if you define edges for the node later on, you can omit text definitions. The one previously defined will be used when rendering the box.

Code:

```
mermaid
graph LR
    id1[This is the text in the box]
```

⌘ + Enter | [Run ▶](#)

Node with text

This is the text in the box

Unicode text

Use `"` to enclose the unicode text.

Code:

```
mermaid
graph LR
    id["This ♥ Unicode"]
```

⌘ + Enter | [Run ▶](#)

This ♥ Unicode

Markdown formatting

Use double quotes and backticks "` text `" to enclose the markdown text.

Code:

mermaid

```
---
config:
  htmlLabels: false
---
flowchart LR
  markdown["`This is _Markdown_`"]
  newLines["`Line1  
Line 2  
Line 3`"]
  markdown --> newLines
```

⌘ + Enter | [Run ▶](#)

Direction

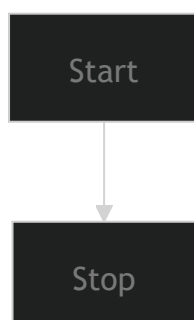
This statement declares the direction of the Flowchart.

This declares the flowchart is oriented from top to bottom (**TD** or **TB**).

Code:

mermaid

```
flowchart TD
  Start --> Stop
```

⌘ + Enter | [Run ▶](#)

This declares the flowchart is oriented from left to right (**LR**).

Code:

mermaid

```
graph LR
    Start --> Stop
```

⌘ + Enter | [Run ▶](#)

Possible FlowChart orientations are:

- TB - Top to bottom
- TD - Top-down/ same as top to bottom
- BT - Bottom to top
- RL - Right to left
- LR - Left to right

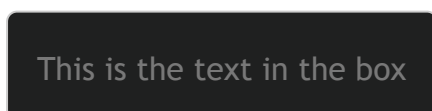
Node shapes

A node with round edges

Code:

mermaid

```
graph LR
    id1([This is the text in the box])
```

⌘ + Enter | [Run ▶](#)

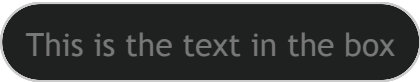
A stadium-shaped node

Code:

mermaid

```
graph LR
    id1([This is the text in the box])
```

⌘ + Enter | [Run ▶](#)



This is the text in the box

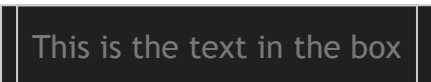
A node in a subroutine shape

Code:

```
graph LR
    id1[This is the text in the box]
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

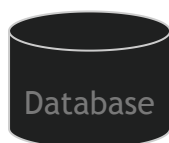
A node in a cylindrical shape

Code:

```
graph LR
    id1[(Database)]
```

mermaid

⌘ + Enter | [Run ▶](#)



A node in the form of a circle

Code:

```
graph LR
    id1((This is the text in the circle))
```

mermaid

⌘ + Enter | [Run ▶](#)



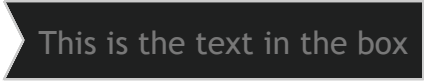
A node in an asymmetric shape

Code:

```
graph LR
    id1>This is the text in the box]
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

Currently only the shape above is possible and not its mirror. *This might change with future releases.*

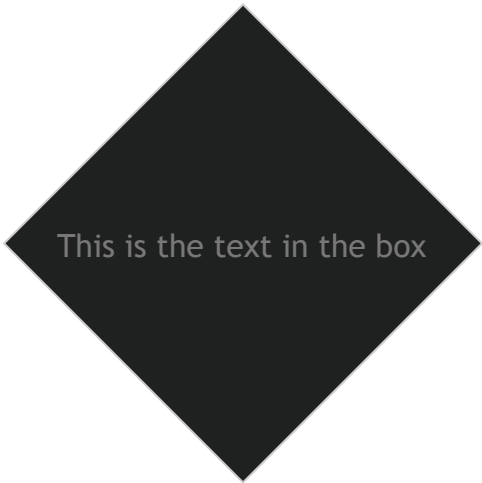
A node (rhombus)

Code:

```
graph LR
    id1{This is the text in the box}
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

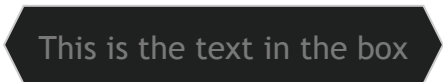
A hexagon node

Code:

```
graph LR
    id1{{This is the text in the box}}
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

Parallelogram

Code:

```
flowchart TD
    id1[/This is the text in the box/]
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

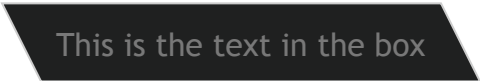
Parallelogram alt

Code:

```
flowchart TD
    id1[\\This is the text in the box\\]
```

mermaid

⌘ + Enter | [Run ▶](#)



This is the text in the box

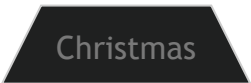
Trapezoid

Code:

```
flowchart TD
    A[/Christmas\\]
```

mermaid

⌘ + Enter | [Run ▶](#)



Christmas

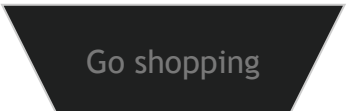
Trapezoid alt

Code:

```
flowchart TD
    B[\\Go shopping/]
```

mermaid

⌘ + Enter | [Run ▶](#)



Go shopping

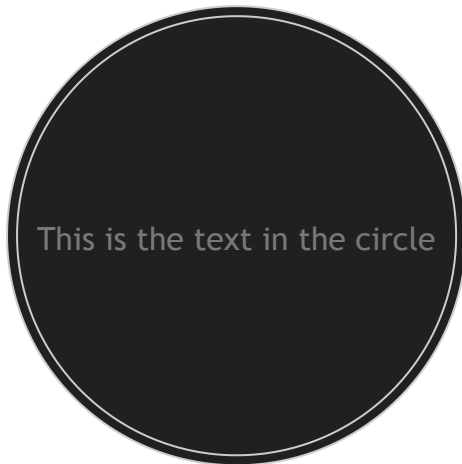
Double circle

Code:

mermaid

```
graph TD
    id1(((This is the text in the circle)))
```

⌘ + Enter | [Run ▶](#)



Expanded Node Shapes in Mermaid Flowcharts (v11.3.0+)

Mermaid introduces 30 new shapes to enhance the flexibility and precision of flowchart creation. These new shapes provide more options to represent processes, decisions, events, data storage visually, and other elements within your flowcharts, improving clarity and semantic meaning.

New Syntax for Shape Definition

Mermaid now supports a general syntax for defining shape types to accommodate the growing number of shapes. This syntax allows you to assign specific shapes to nodes using a clear and flexible format:

```
A@{ shape: rect }
```

This syntax creates a node A as a rectangle. It renders in the same way as **A["A"]** , or **A** .

Complete List of New Shapes

Below is a comprehensive list of the newly introduced shapes and their corresponding semantic meanings, short names, and aliases:

Semantic Name	Shape Name	Short Name	Description	Alias Supported
Bang	Bang	bang	Bang	bang
Card	Notched Rectangle	notch-rect	Represents a card	card , notch-rectangle
Cloud	Cloud	cloud	cloud	cloud
Collate	Hourglass	hourglass	Represents a collate operation	collate , hourglass
Com Link	Lightning Bolt	bolt	Communication link	com-link , lightning-bolt
Comment	Curly Brace	brace	Adds a comment	brace-l , comment
Comment Right	Curly Brace	brace-r	Adds a comment	
Comment with braces on both sides	Curly Braces	braces	Adds a comment	
Data Input/Output	Lean Right	lean-r	Represents input or output	in-out , lean-right
Data Input/Output	Lean Left	lean-l	Represents output or input	lean-left , out-in
Database	Cylinder	cyl	Database storage	cylinder , database , db
Decision	Diamond	diam	Decision-making step	decision , diamond , question
Delay	Half-Rounded Rectangle	delay	Represents a delay	half-rounded-rectangle
Direct Access Storage	Horizontal Cylinder	h-cyl	Direct access storage	das , horizontal-cylinder

Semantic Name	Shape Name	Short Name	Description	Alias Supported
Disk Storage	Lined Cylinder	lin-cyl	Disk storage	disk , lined-cylinder
Display	Curved Trapezoid	curv-trap	Represents a display	curved-trapezoid , display
Divided Process	Divided Rectangle	div-rect	Divided process shape	div-proc , divided-process , divided-rectangle
Document	Document	doc	Represents a document	doc , document
Event	Rounded Rectangle	rounded	Represents an event	event
Extract	Triangle	tri	Extraction process	extract , triangle
Fork/Join	Filled Rectangle	fork	Fork or join in process flow	join
Internal Storage	Window Pane	win-pane	Internal storage	internal-storage , window-pane
Junction	Filled Circle	f-circ	Junction point	filled-circle , junction
Lined Document	Lined Document	lin-doc	Lined document	lined-document
Lined/Shaded Process	Lined Rectangle	lin-rect	Lined process shape	lin-proc , lined-process , lined-rectangle , shaded-process
Loop Limit	Trapezoidal Pentagon	notch-pent	Loop limit step	loop-limit , notched-pentagon
Manual File	Flipped Triangle	flip-tri	Manual file operation	flipped-triangle , manual-file
Manual Input	Sloped Rectangle	sl-rect	Manual input step	manual-input , sloped-rectangle

Semantic Name	Shape Name	Short Name	Description	Alias Supported
Manual Operation	Trapezoid Base Top	trap-t	Represents a manual task	inv-trapezoid , manual , trapezoid-top
Multi-Document	Stacked Document	docs	Multiple documents	documents , st-doc , stacked-document
Multi-Process	Stacked Rectangle	st-rect	Multiple processes	processes , procs , stacked-rectangle
Odd	Odd	odd	Odd shape	
Paper Tape	Flag	flag	Paper tape	paper-tape
Prepare Conditional	Hexagon	hex	Preparation or condition step	hexagon , prepare
Priority Action	Trapezoid Base Bottom	trap-b	Priority action	priority , trapezoid , trapezoid-bottom
Process	Rectangle	rect	Standard process shape	proc , process , rectangle
Start	Circle	circle	Starting point	circ
Start	Small Circle	sm-circ	Small starting point	small-circle , start
Stop	Double Circle	dbl-circ	Represents a stop point	double-circle
Stop	Framed Circle	fr-circ	Stop point	framed-circle , stop
Stored Data	Bow Tie Rectangle	bow-rect	Stored data	bow-tie-rectangle , stored-data
Subprocess	Framed Rectangle	fr-rect	Subprocess	framed-rectangle , subproc , subprocess , subroutine

Semantic Name	Shape Name	Short Name	Description	Alias Supported
Summary	Crossed Circle	cross-circ	Summary	crossed-circle , summary
Tagged Document	Tagged Document	tag-doc	Tagged document	tag-doc , tagged-document
Tagged Process	Tagged Rectangle	tag-rect	Tagged process	tag-proc , tagged-process , tagged-rectangle
Terminal Point	Stadium	stadium	Terminal point	pill , terminal
Text Block	Text Block	text	Text block	

Example Flowchart with New Shapes

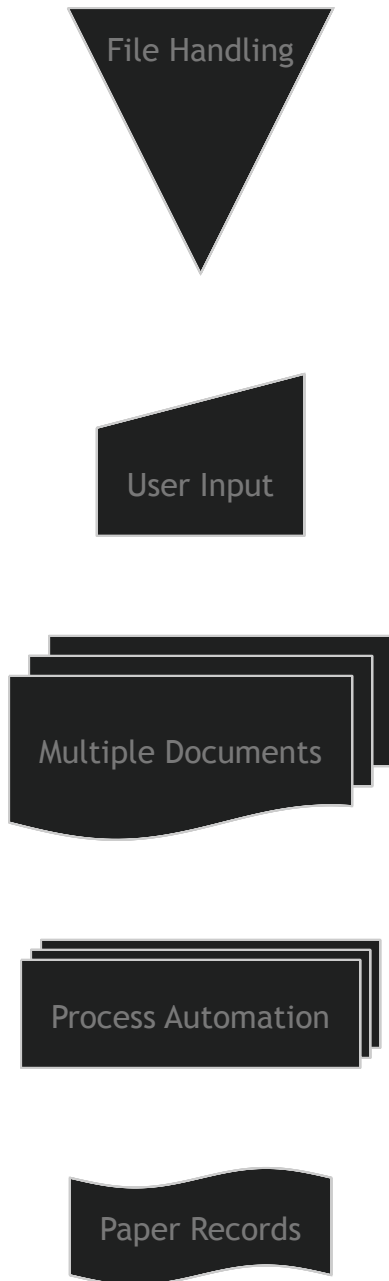
Here’s an example flowchart that utilizes some of the newly introduced shapes:

Code:

mermaid

```
flowchart RL
    A@{ shape: manual-file, label: "File Handling"}
    B@{ shape: manual-input, label: "User Input"}
    C@{ shape: docs, label: "Multiple Documents"}
    D@{ shape: procs, label: "Process Automation"}
    E@{ shape: paper-tape, label: "Paper Records"}
```

⌘ + Enter | Run ▶



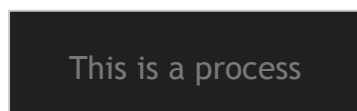
Process

Code:

```
flowchart TD
    A@{ shape: rect, label: "This is a process" }
```

mermaid

⌘ + Enter | [Run ▶](#)



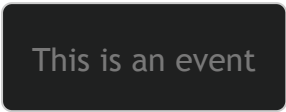
Event

Code:

mermaid

flowchart TD

A@{ shape: rounded, label: "This is an event" }

⌘ + Enter | [Run ►](#)This is an event

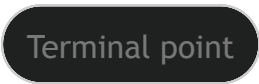
Terminal Point (Stadium)

Code:

mermaid

flowchart TD

A@{ shape: stadium, label: "Terminal point" }

⌘ + Enter | [Run ►](#)Terminal point

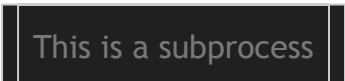
Subprocess

Code:

mermaid

flowchart TD

A@{ shape: subproc, label: "This is a subprocess" }

⌘ + Enter | [Run ►](#)This is a subprocess

Database (Cylinder)

Code:

mermaid

flowchart TD

A@{ shape: cyl, label: "Database" }

⌘ + Enter | [Run ►](#)Database

Start (Circle)

Code:

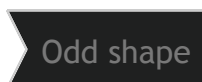
```
graph TD
    A((Start))
```

mermaid

⌘ + Enter | [Run ▶](#)**Odd****Code:**

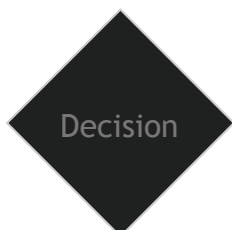
```
graph TD
    A{{Odd shape}}
```

mermaid

⌘ + Enter | [Run ▶](#)**Decision (Diamond)****Code:**

```
graph TD
    A{Decision}
```

mermaid

⌘ + Enter | [Run ▶](#)**Prepare Conditional (Hexagon)****Code:**

```
graph TD
    A{{Prepare conditional}}
```

mermaid

⌘ + Enter | [Run ▶](#)

Prepare conditional

Data Input/Output (Lean Right)

Code:

```
flowchart TD
    A@{ shape: lean-r, label: "Input/Output" }
```

mermaid

⌘ + Enter | [Run ►](#)

Input/Output

Data Input/Output (Lean Left)

Code:

```
flowchart TD
    A@{ shape: lean-l, label: "Output/Input" }
```

mermaid

⌘ + Enter | [Run ►](#)

Output/Input

Priority Action (Trapezoid Base Bottom)

Code:

```
flowchart TD
    A@{ shape: trap-b, label: "Priority action" }
```

mermaid

⌘ + Enter | [Run ►](#)

Priority action

Manual Operation (Trapezoid Base Top)

Code:

```
flowchart TD
    A@{ shape: trap-t, label: "Manual operation" }
```

mermaid

⌘ + Enter | [Run ►](#)

Manual operation

Stop (Double Circle)

Code:

```
flowchart TD
    A@{ shape: dbl-circ, label: "Stop" }
```

mermaid

⌘ + Enter | [Run ▶](#)



Text Block

Code:

```
flowchart TD
    A@{ shape: text, label: "This is a text block" }
```

mermaid

⌘ + Enter | [Run ▶](#)

This is a text block

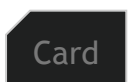
Card (Notched Rectangle)

Code:

```
flowchart TD
    A@{ shape: notch-rect, label: "Card" }
```

mermaid

⌘ + Enter | [Run ▶](#)



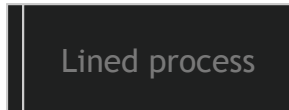
Lined/Shaded Process

Code:

```
flowchart TD
```

mermaid

```
A@{ shape: lin-rect, label: "Lined process" }
```

⌘ + Enter | [Run ►](#)

Start (Small Circle)

Code:

```
flowchart TD
    A@{ shape: sm-circ, label: "Small start" }
```

mermaid

⌘ + Enter | [Run ►](#)

Stop (Framed Circle)

Code:

```
flowchart TD
    A@{ shape: framed-circle, label: "Stop" }
```

mermaid

⌘ + Enter | [Run ►](#)

Fork/Join (Long Rectangle)

Code:

```
flowchart TD
    A@{ shape: fork, label: "Fork or Join" }
```

mermaid

⌘ + Enter | [Run ►](#)

Collate (Hourglass)

Code:

```
flowchart TD
    A@{ shape: hourglass, label: "Collate" }
```

mermaid

⌘ + Enter | [Run ►](#)



Comment (Curly Brace)

Code:

```
flowchart TD
    A@{ shape: comment, label: "Comment" }
```

mermaid

⌘ + Enter | [Run ►](#)



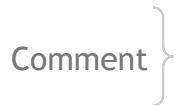
Comment Right (Curly Brace Right)

Code:

```
flowchart TD
    A@{ shape: brace-r, label: "Comment" }
```

mermaid

⌘ + Enter | [Run ►](#)



Comment with braces on both sides

Code:

```
flowchart TD
    A@{ shape: braces, label: "Comment" }
```

mermaid

⌘ + Enter | [Run ►](#)



Com Link (Lightning Bolt)

Code:

```
flowchart TD
    A@{ shape: bolt, label: "Communication link" }
```

mermaid

⌘ + Enter | [Run ►](#)



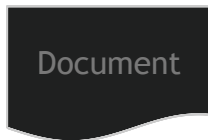
Document

Code:

```
flowchart TD
    A@{ shape: doc, label: "Document" }
```

mermaid

⌘ + Enter | [Run ►](#)



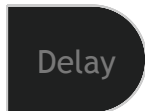
Delay (Half-Rounded Rectangle)

Code:

```
flowchart TD
    A@{ shape: delay, label: "Delay" }
```

mermaid

⌘ + Enter | [Run ►](#)



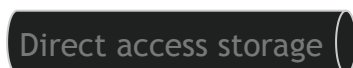
Direct Access Storage (Horizontal Cylinder)

Code:

```
flowchart TD
    A@{ shape: das, label: "Direct access storage" }
```

mermaid

⌘ + Enter | [Run ►](#)



Disk Storage (Lined Cylinder)

Code:

mermaid

flowchart TD

A@{ shape: lin-cyl, label: "Disk storage" }

⌘ + Enter | [Run ►](#)

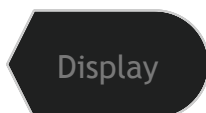
Display (Curved Trapezoid)

Code:

mermaid

flowchart TD

A@{ shape: curv-trap, label: "Display" }

⌘ + Enter | [Run ►](#)

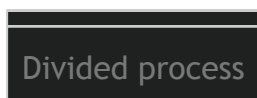
Divided Process (Divided Rectangle)

Code:

mermaid

flowchart TD

A@{ shape: div-rect, label: "Divided process" }

⌘ + Enter | [Run ►](#)

Extract (Small Triangle)

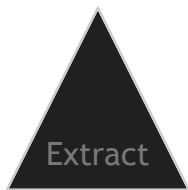
Code:

mermaid

flowchart TD

A@{ shape: tri, label: "Extract" }

⌘ + Enter | [Run ►](#)



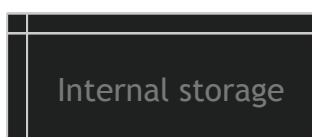
Internal Storage (Window Pane)

Code:

```
flowchart TD
    A@{ shape: win-pane, label: "Internal storage" }
```

mermaid

⌘ + Enter | [Run ►](#)



Junction (Filled Circle)

Code:

```
flowchart TD
    A@{ shape: f-circ, label: "Junction" }
```

mermaid

⌘ + Enter | [Run ►](#)



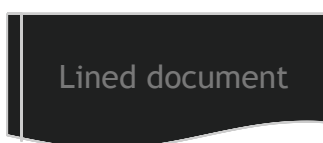
Lined Document

Code:

```
flowchart TD
    A@{ shape: lin-doc, label: "Lined document" }
```

mermaid

⌘ + Enter | [Run ►](#)



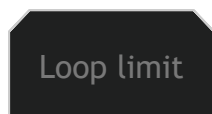
Loop Limit (Notched Pentagon)

Code:

mermaid

flowchart TD

A@{ shape: notch-pent, label: "Loop limit" }

⌘ + Enter | [Run ►](#)

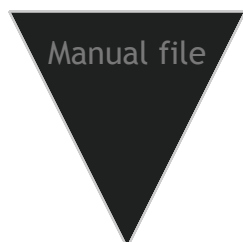
Manual File (Flipped Triangle)

Code:

mermaid

flowchart TD

A@{ shape: flip-tri, label: "Manual file" }

⌘ + Enter | [Run ►](#)

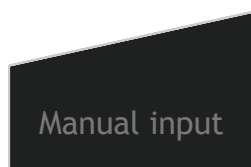
Manual Input (Sloped Rectangle)

Code:

mermaid

flowchart TD

A@{ shape: sl-rect, label: "Manual input" }

⌘ + Enter | [Run ►](#)

Multi-Document (Stacked Document)

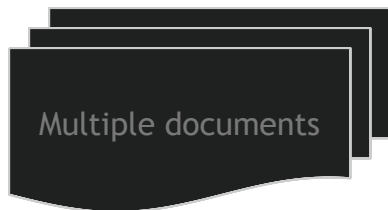
Code:

mermaid

flowchart TD

A@{ shape: docs, label: "Multiple documents" }

⌘ + Enter | [Run ►](#)



Multi-Process (Stacked Rectangle)

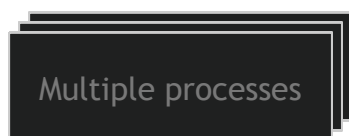
Code:

```
flowchart TD
```

```
A@{ shape: processes, label: "Multiple processes" }
```

mermaid

⌘ + Enter | [Run ►](#)



Paper Tape (Flag)

Code:

```
flowchart TD
```

```
A@{ shape: flag, label: "Paper tape" }
```

mermaid

⌘ + Enter | [Run ►](#)



Stored Data (Bow Tie Rectangle)

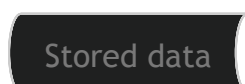
Code:

```
flowchart TD
```

```
A@{ shape: bow-rect, label: "Stored data" }
```

mermaid

⌘ + Enter | [Run ►](#)



Summary (Crossed Circle)

Code:

mermaid

flowchart TD

A@{ shape: cross-circ, label: "Summary" }

⌘ + Enter | [Run ►](#)

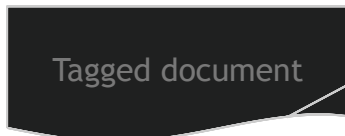
Tagged Document

Code:

flowchart TD

A@{ shape: tag-doc, label: "Tagged document" }

mermaid

⌘ + Enter | [Run ►](#)

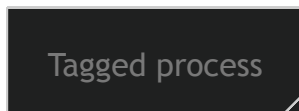
Tagged Process (Tagged Rectangle)

Code:

flowchart TD

A@{ shape: tag-rect, label: "Tagged process" }

mermaid

⌘ + Enter | [Run ►](#)

Special shapes in Mermaid Flowcharts (v11.3.0+)

Mermaid also introduces 2 special shapes to enhance your flowcharts: **icon** and **image**. These shapes allow you to include icons and images directly within your flowcharts, providing more visual context and clarity.

Icon Shape

You can use the **icon** shape to include an icon in your flowchart. To use icons, you need to register the icon pack first. Follow the instructions to [add custom icons](#). The syntax for defining an icon shape is as follows:

Code:

```

flowchart TD
    A@{ icon: "fa:user", form: "square", label: "User Icon", pos: "t", h:
  
```

mermaid

⌘ + Enter | [Run ▶](#)

User Icon



Parameters

- **icon:** The name of the icon from the registered icon pack.
- **form:** Specifies the background shape of the icon. If not defined there will be no background to icon. Options include:
 - **square**
 - **circle**
 - **rounded**
- **label:** The text label associated with the icon. This can be any string. If not defined, no label will be displayed.
- **pos:** The position of the label. If not defined label will default to bottom of icon. Possible values are:
 - **t**
 - **b**
- **h:** The height of the icon. If not defined this will default to 48 which is minimum.

Image Shape

You can use the **image** shape to include an image in your flowchart. The syntax for defining an image shape is as follows:

```
flowchart TD
```

```
A@{ img: "https://example.com/image.png", label: "Image Label", pos:
```

Parameters

- **img**: The URL of the image to be displayed.
- **label**: The text label associated with the image. This can be any string. If not defined, no label will be displayed.
- **pos**: The position of the label. If not defined, the label will default to the bottom of the image. Possible values are:
 - **t**
 - **b**
- **w**: The width of the image. If not defined, this will default to the natural width of the image.
- **h**: The height of the image. If not defined, this will default to the natural height of the image.
- **constraint**: Determines if the image should constrain the node size. This setting also ensures the image maintains its original aspect ratio, adjusting the width (**w**) accordingly to the height (**h**). If not defined, this will default to **off** Possible values are:
 - **on**
 - **off**

If you want to resize an image, but keep the same aspect ratio, set **h** , and set **constraint: on** to constrain the aspect ratio. E.g.

Code:

```
flowchart TD
```

```
%% My image with a constrained aspect ratio
```

```
A@{ img: "https://mermaid.js.org/favicon.svg", label: "My example image
```

mermaid

⌘ + Enter | **Run ▶**

My example image label



Links between nodes

Nodes can be connected with links/edges. It is possible to have different types of links or attach a text string to a link.

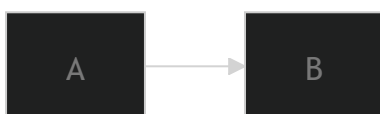
A link with arrow head

Code:

```
graph LR
  A-->B
```

mermaid

⌘ + Enter | [Run ▶](#)



An open link

Code:

```
graph LR
  A --- B
```

mermaid

⌘ + Enter | [Run ▶](#)



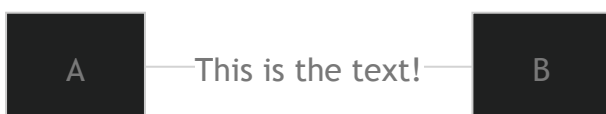
Text on links

Code:

```
graph LR
  A -- "This is the text!" --- B
```

mermaid

⌘ + Enter | [Run ▶](#)



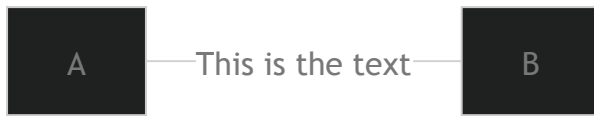
or

Code:

mermaid

flowchart LR

A---|This is the text|B

⌘ + Enter | [Run ►](#)

A link with arrow head and text

Code:

mermaid

flowchart LR

A-->|text|B

⌘ + Enter | [Run ►](#)

or

Code:

mermaid

flowchart LR

A-- text -->B

⌘ + Enter | [Run ►](#)

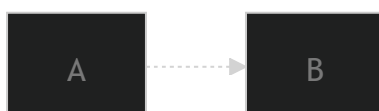
Dotted link

Code:

mermaid

flowchart LR

A-.->B;

⌘ + Enter | [Run ►](#)

Dotted link with text

Code:

```
graph LR
  A-. text .-> B
```

mermaid

⌘ + Enter | [Run ▶](#)



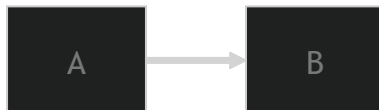
Thick link

Code:

```
graph LR
  A ==> B
```

mermaid

⌘ + Enter | [Run ▶](#)



Thick link with text

Code:

```
graph LR
  A == text ==> B
```

mermaid

⌘ + Enter | [Run ▶](#)



An invisible link

This can be a useful tool in some instances where you want to alter the default positioning of a node.

Code:

```
graph LR
```

mermaid

A ~~~ B

⌘ + Enter | [Run ▶](#)



Chaining of links

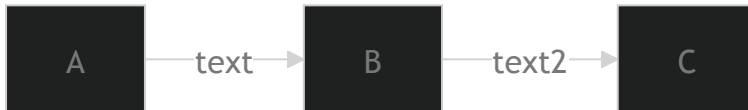
It is possible declare many links in the same line as per below:

Code:

```
flowchart LR
  A -- text --> B -- text2 --> C
```

mermaid

⌘ + Enter | [Run ▶](#)



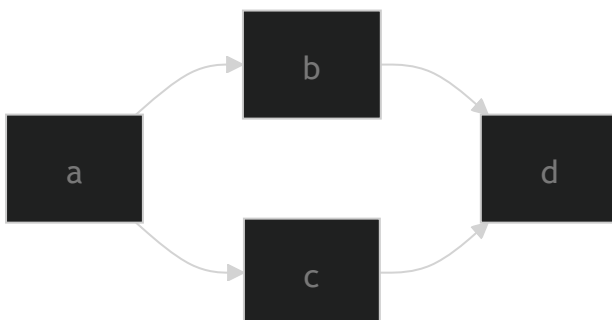
It is also possible to declare multiple nodes links in the same line as per below:

Code:

```
flowchart LR
  a --> b & c --> d
```

mermaid

⌘ + Enter | [Run ▶](#)



You can then describe dependencies in a very expressive way. Like the one-liner below:

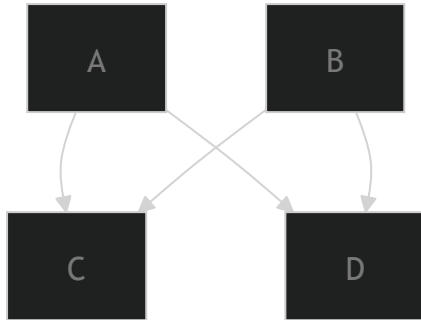
Code:

```
flowchart TB
```

mermaid

A & B--> C & D

⌘ + Enter | [Run ▶](#)



If you describe the same diagram using the basic syntax, it will take four lines. A word of warning, one could go overboard with this making the flowchart harder to read in markdown form. The Swedish word **lagom** comes to mind. It means, not too much and not too little. This goes for expressive syntaxes as well.

Code:

flowchart TB

A --> C

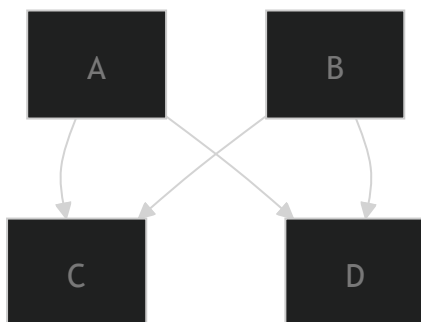
A --> D

B --> C

B --> D

mermaid

⌘ + Enter | [Run ▶](#)



Attaching an ID to Edges

Mermaid now supports assigning IDs to edges, similar to how IDs and metadata can be attached to nodes. This feature lays the groundwork for more advanced styling, classes, and animation capabilities on edges.

Syntax:

To give an edge an ID, prepend the edge syntax with the ID followed by an **@** character. For example:

Code:

mermaid

```
graph LR
  A -- e1 --> B
```

⌘ + Enter | [Run ▶](#)

In this example, **e1** is the ID of the edge connecting **A** to **B**. You can then use this ID in later definitions or style statements, just like with nodes.

Turning an Animation On

Once you have assigned an ID to an edge, you can turn on animations for that edge by defining the edge's properties:

Code:

mermaid

```
graph LR
  A -- e1 ==> B
  e1 --> B { animate: true }
```

⌘ + Enter | [Run ▶](#)

This tells Mermaid that the edge **e1** should be animated.

Selecting Type of Animation

In the initial version, two animation speeds are supported: **fast** and **slow**.

Selecting a specific animation type is a shorthand for enabling animation and setting the animation speed in one go.

Examples:**Code:**

mermaid

```
graph LR
  A -- e1 --> B
```

```
e1@{ animation: fast }
```

⌘ + Enter | [Run ▶](#)

This is equivalent to `{ animate: true, animation: fast }`.

Using classDef Statements for Animations

You can also animate edges by assigning a class to them and then defining animation properties in a `classDef` statement. For example:

Code:

```
flowchart LR
  A e1@--> B
  classDef animate stroke-dasharray: 9,5,stroke-dashoffset: 900,animation
  class e1 animate
```

mermaid

⌘ + Enter | [Run ▶](#)

In this snippet:

- `e1@-->` creates an edge with ID `e1`.
- `classDef animate` defines a class named `animate` with styling and animation properties.
- `class e1 animate` applies the `animate` class to the edge `e1`.

Note on Escaping Commas: When setting the `stroke-dasharray` property, remember to escape commas as `\,` since commas are used as delimiters in Mermaid's style definitions.

New arrow types

There are new types of arrows supported:

- circle edge
- cross edge

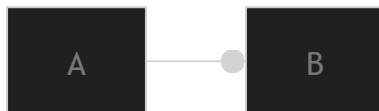
Circle edge example

Code:

```
graph LR
  A --o B
```

mermaid

⌘ + Enter | [Run ▶](#)



Cross edge example

Code:

```
graph LR
  A --x B
```

mermaid

⌘ + Enter | [Run ▶](#)



Multi directional arrows

There is the possibility to use multidirectional arrows.

Code:

```
graph LR
  A o--o B
  B <--> C
  C x--x D
```

mermaid

⌘ + Enter | [Run ▶](#)



Minimum length of a link

Each node in the flowchart is ultimately assigned to a rank in the rendered graph, i.e. to a vertical or horizontal level (depending on the flowchart orientation), based on the nodes to which it is linked. By default, links can span any number of ranks, but you can ask for any link to be longer than the others by adding extra dashes in the link definition.

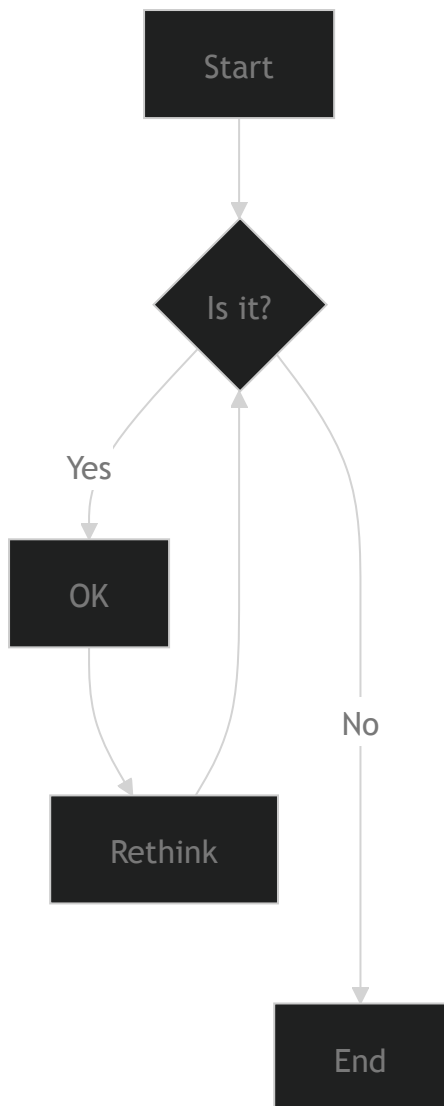
In the following example, two extra dashes are added in the link from node *B* to node *E*, so that it spans two more ranks than regular links:

Code:

```
flowchart TD
    A[Start] --> B{Is it?}
    B -->|Yes| C[OK]
    C --> D[Rethink]
    D --> B
    B ---->|No| E[End]
```

mermaid

⌘ + Enter | [Run ▶](#)



Note Links may still be made longer than the requested number of ranks by the rendering engine to accommodate other requests.

When the link label is written in the middle of the link, the extra dashes must be added on the right side of the link. The following example is equivalent to the previous one:

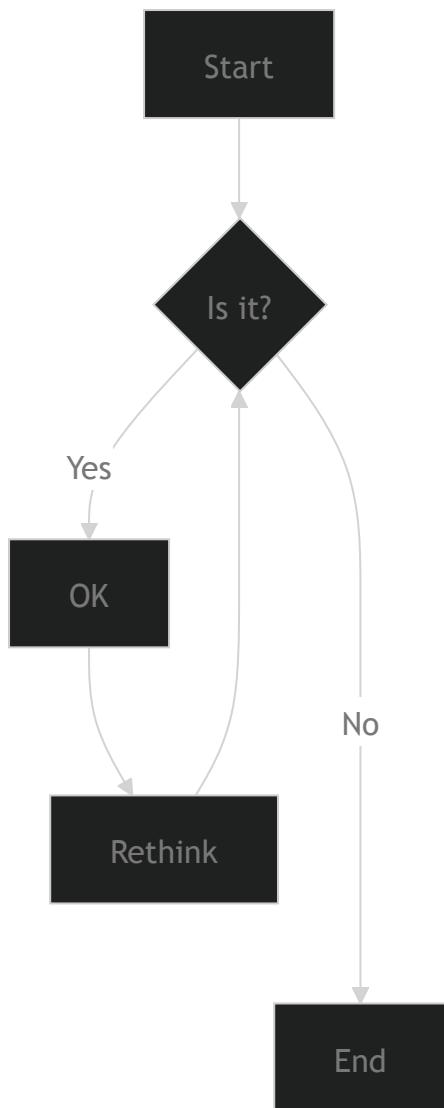
Code:

```

flowchart TD
    A[Start] --> B{Is it?}
    B -- Yes --> C[OK]
    C --> D[Rethink]
    D --> B
    B -- No -----> E[End]
  
```

mermaid

⌘ + Enter | [Run ▶](#)



For dotted or thick links, the characters to add are equals signs or dots, as summed up in the following table:

Length	1	2	3
Normal	---	----	-----
Normal with arrow	-->	---->	----->
Thick	===	=====	=====
Thick with arrow	==>	=====>	=====>
Dotted	-.-	-.-.-	-.-.-.-
Dotted with arrow	-.->	-.-.->	-.-.-.->

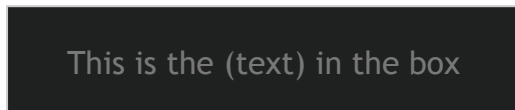
Special characters that break syntax

It is possible to put text within quotes in order to render more troublesome characters. As in the example below:

Code:

```
flowchart LR
    id1["This is the (text) in the box"]
```

mermaid

⌘ + Enter | [Run ▶](#)

Entity codes to escape characters

It is possible to escape characters using the syntax exemplified here.

Code:

```
flowchart LR
    A["A double quote:#quot;"] --> B["A dec char:#9829;"]
```

mermaid

⌘ + Enter | [Run ▶](#)

Numbers given are base 10, so **#** can be encoded as **#35;** . It is also supported to use HTML character names.

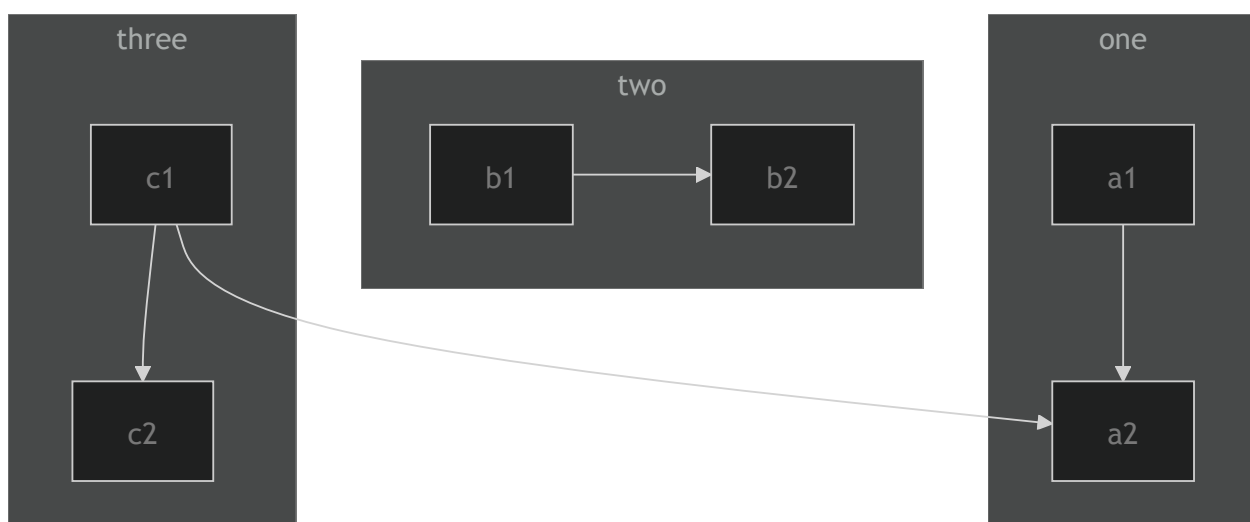
Subgraphs

```
subgraph title
    graph definition
end
```

An example below:

Code:

```
graph TD
    c1-->a2
    subgraph one
        a1-->a2
    end
    subgraph two
        b1-->b2
    end
    subgraph three
        c1-->c2
    end
```

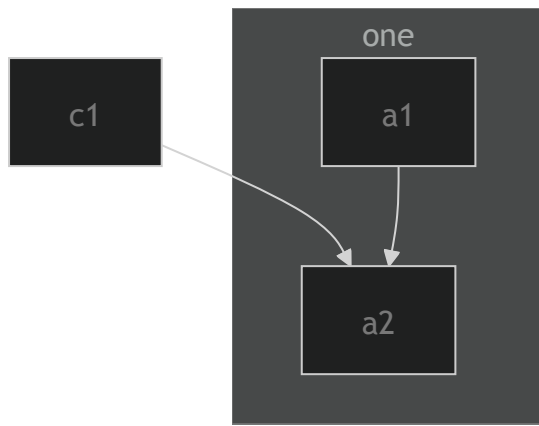
⌘ + Enter | [Run ▶](#)

You can also set an explicit id for the subgraph.

Code:

```
graph TD
    c1-->a2
    subgraph ide1 [one]
        a1-->a2
    end
```

⌘ + Enter | [Run ▶](#)



flowcharts

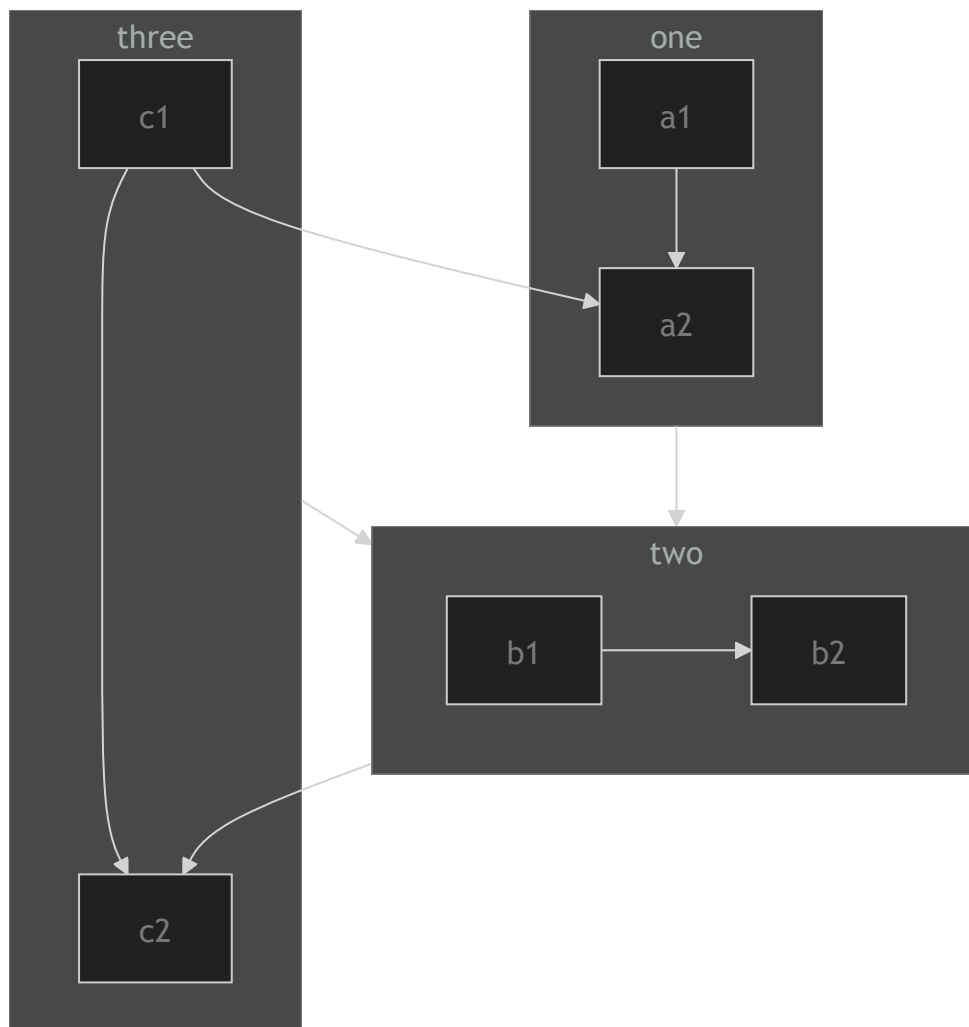
With the graphtype flowchart it is also possible to set edges to and from subgraphs as in the flowchart below.

Code:

```
flowchart TB
    c1-->a2
    subgraph one
        a1-->a2
    end
    subgraph two
        b1-->b2
    end
    subgraph three
        c1-->c2
    end
    one --> two
    three --> two
    two --> c2
```

mermaid

⌘ + Enter | [Run ▶](#)



Direction in subgraphs

With the graphtype flowcharts you can use the direction statement to set the direction which the subgraph will render like in this example.

Code:

```

flowchart LR
    subgraph TOP
        direction TB
        subgraph B1
            direction RL
            i1 --> f1
        end
        subgraph B2
            direction BT
            i2 --> f2
        end
    end
end

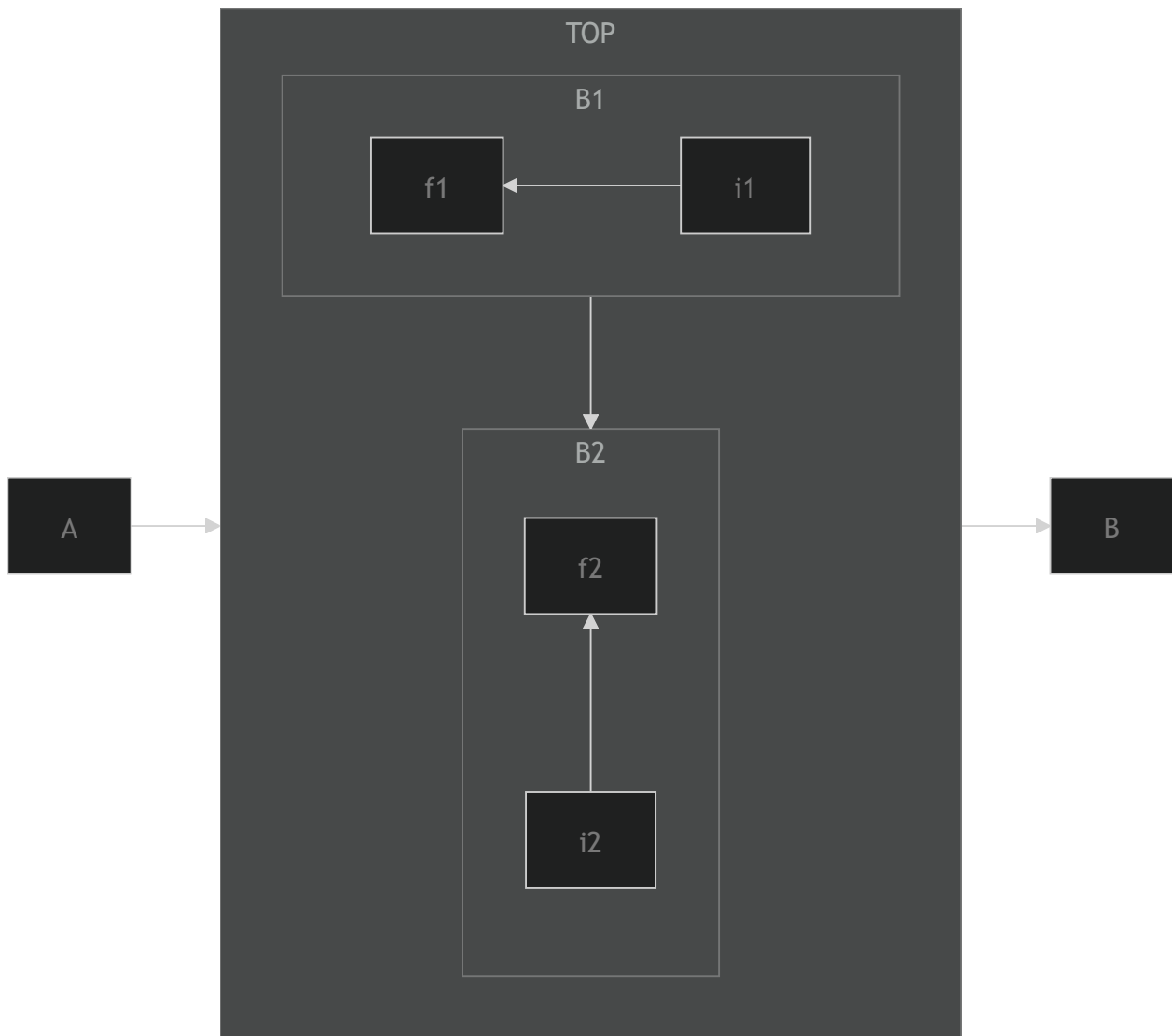
```

mermaid

A --> TOP --> B

B1 --> B2

⌘ + Enter | [Run ▶](#)



Limitation

If any of a subgraph's nodes are linked to the outside, subgraph direction will be ignored. Instead the subgraph will inherit the direction of the parent graph:

Code:

```

flowchart LR
    subgraph subgraph1
        direction TB
        top1[top] --> bottom1[bottom]
    end
    subgraph subgraph2
        direction TB
        top2[top] --> bottom2[bottom]
    end

```

mermaid

```
end
```

```
%% ^ These subgraphs are identical, except for the links to them:
```

```
%% Link *to* subgraph1: subgraph1 direction is maintained
```

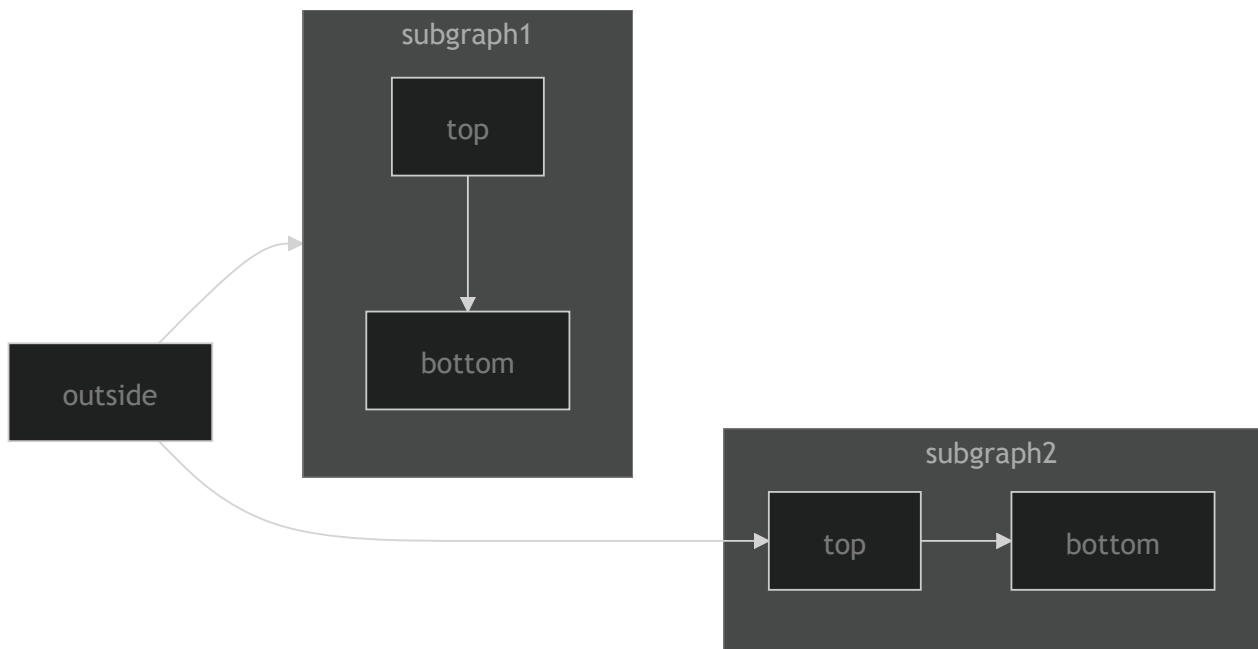
```
outside --> subgraph1
```

```
%% Link *within* subgraph2:
```

```
%% subgraph2 inherits the direction of the top-level graph (LR)
```

```
outside ---> top2
```

⌘ + Enter | [Run ▶](#)



Markdown Strings

The "Markdown Strings" feature enhances flowcharts and mind maps by offering a more versatile string type, which supports text formatting options such as bold and italics, and automatically wraps text within labels.

Code:

```

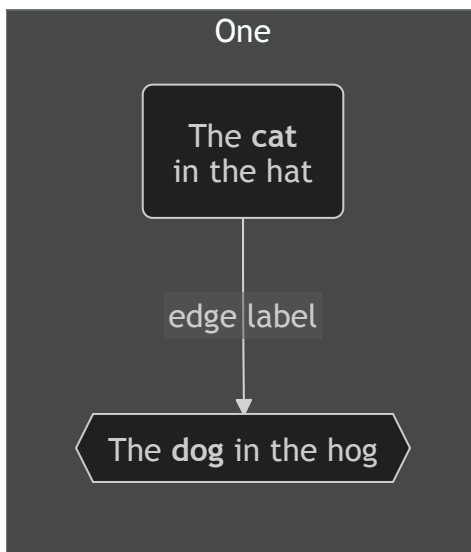
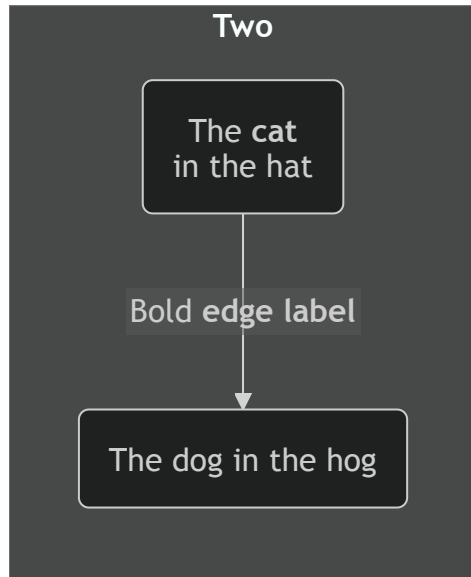
mermaid
----
config:
  htmlLabels: false
----
flowchart LR
  subgraph "One"
    a("`The cat in the hat`") -- "edge label" --> b("`The dog in the hog`")
  end
  
```

```

end
subgraph "`**Two**`"
  c("`The cat  
in the hat`") -- "`edge label`" --> d("The dog in the hog")
end

```

⌘ + Enter | [Run ▶](#)



Formatting:

- For bold text, use double asterisks (******) before and after the text.
- For italics, use single asterisks (***) before and after the text.
- With traditional strings, you needed to add `<br>` tags for text to wrap in nodes. However, markdown strings automatically wrap text when it becomes too long and allows you to start a new line by simply using a newline character instead of a `<br>` tag.

This feature is applicable to node labels, edge labels, and subgraph labels.

The auto wrapping can be disabled by using

```
---  
config:  
  markdownAutoWrap: false  
---  
graph LR
```

Interaction

It is possible to bind a click event to a node, the click can lead to either a javascript callback or to a link which will be opened in a new browser tab.

INFO

This functionality is disabled when using `securityLevel='strict'` and enabled when using `securityLevel='loose'` .

```
click nodeId callback  
click nodeId call callback()
```

- `nodeId` is the id of the node
- `callback` is the name of a javascript function defined on the page displaying the graph, the function will be called with the `nodeId` as parameter.

Examples of tooltip usage below:

```
<script>  
  window.callback = function () {  
    alert('A callback was triggered');  
  };  
</script>
```

html

The tooltip text is surrounded in double quotes. The styles of the tooltip are set by the class `.mermaidTooltip` .

Code:

mermaid

```

flowchart LR
    A-->B
    B-->C
    C-->D
    click A callback "Tooltip for a callback"
    click B "https://www.github.com" "This is a tooltip for a link"
    click C call callback() "Tooltip for a callback"
    click D href "https://www.github.com" "This is a tooltip for a link"

```

⌘ + Enter | **Run ▶**

Success The tooltip functionality and the ability to link to urls are available from version 0.5.2.

?> Due to limitations with how Docsify handles JavaScript callback functions, an alternate working demo for the above code can be viewed at [this jsfiddle](#).

Links are opened in the same browser tab/window by default. It is possible to change this by adding a link target to the click definition (`_self` , `_blank` , `_parent` and `_top` are supported):

Code:

mermaid

```

flowchart LR
    A-->B
    B-->C
    C-->D
    D-->E
    click A "https://www.github.com" _blank
    click B "https://www.github.com" "Open this in a new tab" _blank
    click C href "https://www.github.com" _blank
    click D href "https://www.github.com" "Open this in a new tab" _blank

```

⌘ + Enter | **Run ▶**

Beginner's tip—a full example using interactive links in a html context:

html

```
<body>
  <pre class="mermaid">
    flowchart LR
      A-->B
      B-->C
      C-->D
      click A callback "Tooltip"
      click B "https://www.github.com" "This is a link"
      click C call callback() "Tooltip"
      click D href "https://www.github.com" "This is a link"
  </pre>

  <script>
    window.callback = function () {
      alert('A callback was triggered');
    };
    const config = {
      startOnLoad: true,
      htmlLabels: true,
      flowchart: { useMaxWidth: true, curve: 'cardinal' },
      securityLevel: 'loose',
    };
    mermaid.initialize(config);
  </script>
</body>
```

Comments

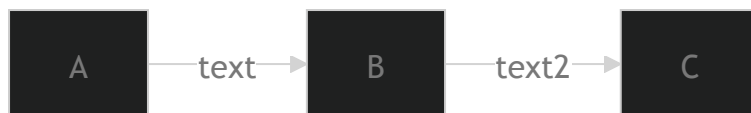
Comments can be entered within a flow diagram, which will be ignored by the parser. Comments need to be on their own line, and must be prefaced with `%%` (double percent signs). Any text after the start of the comment to the next newline will be treated as a comment, including any flow syntax

Code:

mermaid

```
flowchart LR
  %% this is a comment A -- text --> B{node}
  A -- text --> B -- text2 --> C
```

⌘ + Enter | [Run ▶](#)



Styling and classes

Styling links

It is possible to style links. For instance, you might want to style a link that is going backwards in the flow. As links have no ids in the same way as nodes, some other way of deciding what style the links should be attached to is required. Instead of ids, the order number of when the link was defined in the graph is used, or use default to apply to all links. In the example below the style defined in the linkStyle statement will belong to the fourth link in the graph:

```
linkStyle 3 stroke:#ff3,stroke-width:4px,color:red;
```

It is also possible to add style to multiple links in a single statement, by separating link numbers with commas:

```
linkStyle 1,2,7 color:blue;
```

Styling line curves

It is possible to style the type of curve used for lines between items, if the default method does not meet your needs. Available curve styles include `basis` , `bumpX` , `bumpY` , `cardinal` , `catmullRom` , `linear` , `monotoneX` , `monotoneY` , `natural` , `step` , `stepAfter` , and `stepBefore` .

For a full list of available curves, including an explanation of custom curves, refer to the [Shapes](#) documentation in the [d3-shape](#) project.

Line styling can be achieved in two ways:

1. Change the curve style of all the lines
2. Change the curve style of a particular line

Diagram level curve style

In this example, a left-to-right graph uses the **stepBefore** curve style:

```

---
config:
  flowchart:
    curve: stepBefore
---
graph LR

```

Edge level curve style using Edge IDs (v11.10.0+)

You can assign IDs to edges. After assigning an ID you can modify the line style by modifying the edge's **curve** property using the following syntax:

Code:

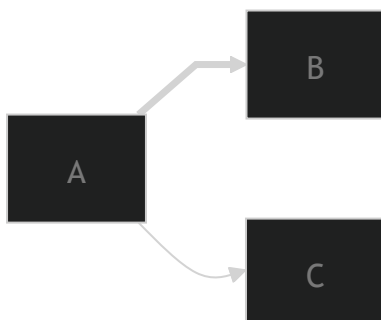
```

flowchart LR
  A e1@==> B
  A e2@--> C
  e1@{ curve: linear }
  e2@{ curve: natural }

```

mermaid

⌘ + Enter | [Run ▶](#)



info

Any edge curve style modified at the edge level overrides the diagram level

info

If the same edge is modified multiple times the last modification will be

Styling a node

It is possible to apply specific styles such as a thicker border or a different background color to a node.

Code:

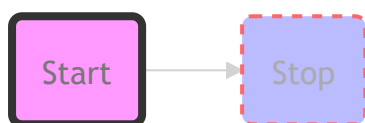
```
flowchart LR
```

```
  id1(Start)-->id2(Stop)
```

```
  style id1 fill:#f9f,stroke:#333,stroke-width:4px
```

```
  style id2 fill:#bbf,stroke:#f66,stroke-width:2px,color:#fff,stroke-da
```

mermaid

⌘ + Enter | [Run ▶](#)

Classes

More convenient than defining the style every time is to define a class of styles and attach this class to the nodes that should have a different look.

A class definition looks like the example below:

```
classDef className fill:#f9f,stroke:#333,stroke-width:4px;
```

Also, it is possible to define style to multiple classes in one statement:

```
classDef firstClassName,secondClassName font-size:12pt;
```

Attachment of a class to a node is done as per below:

```
class nodeId1 className;
```

It is also possible to attach a class to a list of nodes in one statement:

```
class nodeId1,nodeId2 className;
```

A shorter form of adding a class is to attach the classname to the node using the **:::** operator as per below:

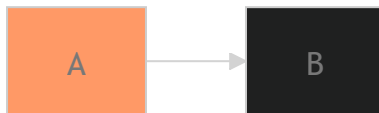
Code:

mermaid

```

flowchart LR
  A:::someclass --> B
  classDef someclass fill:#f96

```

⌘ + Enter | [Run ▶](#)

This form can be used when declaring multiple links between nodes:

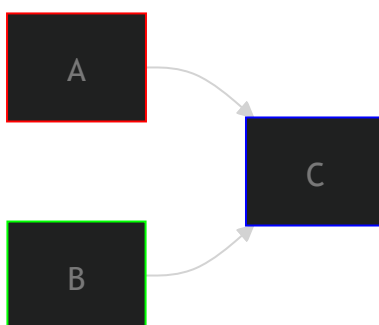
Code:

mermaid

```

flowchart LR
  A:::foo & B:::bar --> C:::foobar
  classDef foo stroke:#f00
  classDef bar stroke:#0f0
  classDef foobar stroke:#00f

```

⌘ + Enter | [Run ▶](#)**CSS classes**

It is also possible to predefine classes in CSS styles that can be applied from the graph definition as in the example below:

Example style

html

```

<style>
  .cssClass > rect {
    fill: #ff0000;
    stroke: #ffff00;
    stroke-width: 4px;

```

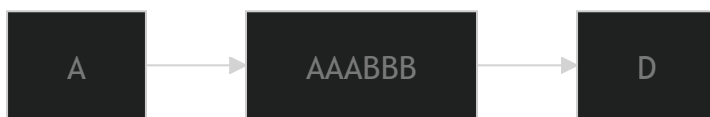
```
}
</style>
```

Example definition

Code:

```
flowchart LR
    A-->B[AAA<span>BBB</span>]
    B-->D
    class A cssClass
```

mermaid

⌘ + Enter | [Run ▶](#)

Default class

If a class is named default it will be assigned to all classes without specific class definitions.

```
classDef default fill:#f9f,stroke:#333,stroke-width:4px;
```

Basic support for fontawesome

It is possible to add icons from fontawesome.

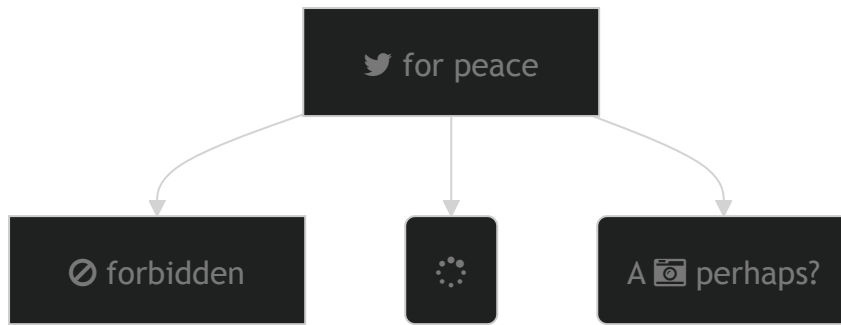
The icons are accessed via the syntax `fa:#icon class name#`.

Code:

```
flowchart TD
    B["fa:fa-twitter for peace"]
    B-->C[fa:fa-ban forbidden]
    B-->D(fa:fa-spinner)
    B-->E(A fa:fa-camera-retro perhaps?)
```

mermaid

⌘ + Enter | [Run ▶](#)



Note that this is currently a paid feature from Font Awesome.

For custom icons, you need to use the **fak** prefix.

Example

```
flowchart TD
    B[fa:fa-twitter] %% standard icon
    B-->E(fak:fa-custom-icon-name) %% custom icon
```

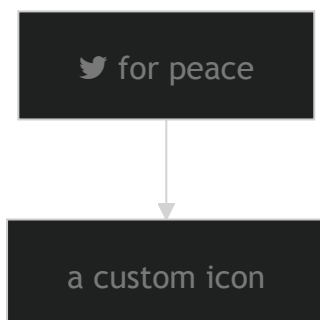
And trying to render it

Code:

```
flowchart TD
    B["fa:fa-twitter for peace"]
    B-->C["fab:fa-truck-bold a custom icon"]
```

mermaid

⌘ + Enter | [Run ▶](#)



Graph declarations with spaces between vertices and link and without semicolon

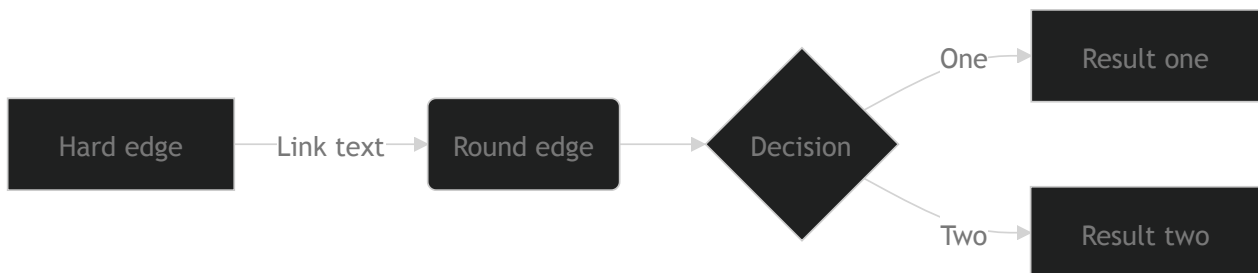
- In graph declarations, the statements also can now end without a semicolon. After release 0.2.16, ending a graph statement with semicolon is just optional. So the below graph declaration is also valid along with the old declarations of the graph.
- A single space is allowed between vertices and the link. However there should not be any space between a vertex and its text and a link and its text. The old syntax of graph declaration will also work and hence this new feature is optional and is introduced to improve readability.

Below is the new declaration of the graph edges which is also valid along with the old declaration of the graph edges.

Code:

```
flowchart LR
    A[Hard edge] -->|Link text| B(Round edge)
    B --> C{Decision}
    C -->|One| D[Result one]
    C -->|Two| E[Result two]
```

mermaid

⌘ + Enter | [Run ▶](#)

Configuration

Renderer

The layout of the diagram is done with the renderer. The default renderer is dagre.

Starting with Mermaid version 9.4, you can use an alternate renderer named elk. The elk renderer is better for larger and/or more complex diagrams.

The *elk* renderer is an experimental feature. You can change the renderer to elk by adding this directive:

```
config:
  flowchart:
    defaultRenderer: "elk"
```

INFO

Note that the site needs to use mermaid version 9.4+ for this to work and have this featured enabled in the lazy-loading configuration.

Width

It is possible to adjust the width of the rendered flowchart.

This is done by defining **mermaid.flowchartConfig** or by the CLI to use a JSON file with the configuration. How to use the CLI is described in the mermaidCLI page.

mermaid.flowchartConfig can be set to a JSON string with config parameters or the corresponding object.

```
mermaid.flowchartConfig = {  
  width: 100%  
}
```

javascript

[Edit this page on GitHub](#)

Previous page

[Syntax and Configuration](#)

Next page

[Sequence Diagram](#)